

Name : Adhi Kesavan S

Reg No : 192521172

Course : Computer Vision

Course Code : ITA0503

CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

SOLUTION :

1. Real-Time Face Detection and Recognition

Aim

To design an OpenCV-based real-time face detection and recognition system that continuously captures frames from a camera, detects multiple faces, extracts facial features, recognizes

registered persons, and displays their identity using bounding boxes while maintaining low processing delay.

Pseudocode :

ALGORITHM: Real-Time Face Detection and Recognition

INPUT:

Live camera/video stream

Trained face recognition dataset/model

OUTPUT:

Video frame with face bounding boxes and identity labels

BEGIN

1. INITIALIZATION

Import OpenCV and required libraries.

Load the trained face recognition model.

Load the registered persons' face feature data
and corresponding names/IDs.

Load the face detection model
(Haar Cascade / OpenCV DNN).

Open the camera using VideoCapture.

IF camera cannot be opened THEN

 Display "Camera Error"

 STOP

END IF

2. CONTINUOUS VIDEO ACQUISITION

WHILE camera is open DO

 Capture one frame from the camera.

 IF frame is not successfully captured THEN

BREAK
END IF

3. PREPROCESSING

Resize the frame to a suitable resolution
to reduce processing time.
Convert the frame to grayscale
if the selected face detector requires grayscale.
Apply Gaussian filtering if necessary
to reduce camera noise.
Improve contrast using histogram equalization
or CLAHE when lighting conditions are poor.

4. FACE DETECTION

Pass the preprocessed frame to
the face detection algorithm.
Detect all faces present in the frame.
FOR each detected face DO
 Obtain the bounding box:
 x, y, width, height
 Extract the face region from the frame.

5. FACE PREPROCESSING AND ALIGNMENT

Crop the detected face.
Resize the cropped face to the
input size required by the recognition model.
Normalize the face image.
If facial landmarks are available,

align the face using eyes/nose landmarks.

6. FEATURE EXTRACTION

Pass the processed face to
the trained feature extraction model.

Generate a facial feature vector
or face embedding.

7. FACE RECOGNITION

Compare the extracted feature vector
with stored registered-person features.
Calculate similarity/distance.
Find the closest matching registered person.
IF similarity is greater than
the predefined recognition threshold THEN
identity = matched person's name/ID
ELSE
identity = "Unknown"
END IF

8. VISUALIZATION

Draw a rectangle around the detected face.
Display the person's identity
above or below the bounding box.
Display confidence/similarity score
if available.
END FOR

9. DISPLAY RESULT

Display the processed frame
using cv2.imshow().

10. EFFICIENT LOOP CONTROL

Check keyboard input.
IF user presses 'q' or ESC THEN
 BREAK
END IF
Continue processing the next frame.

11. RESOURCE RELEASE

Release the camera.
Destroy all OpenCV windows.
END

2. Vehicle Detection, Tracking and Counting

Aim

To design an OpenCV-based traffic monitoring system that detects multiple vehicles inside a predefined region, tracks them across consecutive frames, counts vehicles when they cross a virtual line, and displays bounding boxes, tracking IDs and the total vehicle count.

Pseudocode

ALGORITHM: Vehicle Detection, Tracking and Counting

INPUT:

Live traffic camera/video stream

OUTPUT:

Detected vehicles with bounding boxes, tracking IDs and vehicle count

BEGIN

1. INITIALIZATION

Import OpenCV and required libraries.

Open the traffic video/camera using VideoCapture.

IF camera/video cannot be opened THEN

 Display "Video Error"

 STOP

END IF

Define the Region of Interest (ROI).

Define the virtual counting line.

Initialize vehicle detector.

Initialize object tracker.

Initialize:

 vehicle_count = 0

Create an empty set/list:

 counted_vehicle_ids

Set minimum vehicle detection area.

2. VIDEO PROCESSING LOOP

WHILE video is available DO

 Read one frame.

 IF frame cannot be read THEN

 BREAK

 END IF

3. PREPROCESSING

Resize the frame if necessary.

Apply Gaussian blur to reduce camera noise.

Convert to grayscale if required
by the selected detection method.
Extract only the predefined ROI.

4. VEHICLE DETECTION

Apply the vehicle detection algorithm.

Detect vehicles inside the ROI.

FOR every detected vehicle DO

Obtain:

x, y, width, height

Calculate the vehicle centroid:

$\text{center_x} = x + \text{width} / 2$

$\text{center_y} = y + \text{height} / 2$

Store the bounding box and centroid.

END FOR

5. OBJECT TRACKING

Send the current vehicle detections
to the object tracker.

Associate current detections
with previously detected vehicles.

Assign a unique tracking ID
to every vehicle.

Example:

Vehicle 1 → ID 1

Vehicle 2 → ID 2

Vehicle 3 → ID 3

Update each vehicle's position
across consecutive frames.

6. LINE-CROSSING DETECTION

```
FOR each tracked vehicle DO
    Obtain its current centroid.
    Obtain its previous centroid.
    IF the vehicle moved from one side
      of the virtual line to the other THEN
        IF vehicle ID is NOT in
          counted_vehicle_ids THEN
            vehicle_count =
              vehicle_count + 1
            Add vehicle ID to
              counted_vehicle_ids
          END IF
        END IF
      END IF
    END FOR
```

7. VISUALIZATION

```
Draw the predefined ROI.
Draw the virtual counting line.
FOR each tracked vehicle DO
    Draw its bounding box.
```


Display its tracking ID.
Draw its centroid.
END FOR
Display total vehicle count.

8. DISPLAY FRAME

Show the processed video frame
using cv2.imshow().

9. EFFICIENT LOOP TERMINATION

Check keyboard input.
IF user presses 'q' or ESC THEN
 BREAK
END IF
Continue with the next frame.

10. RESOURCE RELEASE

Release the video/camera.
Destroy all OpenCV windows.
END